
VIABILITY-GATED FEEDFORWARD COMPENSATION OF LEARNED CONTINUOUS-CONTROL POLICIES

A PREPRINT

Zongtan Li

zongtanli2023@gmail.com

July 2026

ABSTRACT

Learned continuous-control policies are rarely uniformly good: even a converged reinforcement-learning controller retains systematic deficiencies such as energy-inefficient command tracking, undertrained command regions, and posture degradation on long tasks. We study *compensation*: improving a fixed learned action policy using its own behavioral experience, without retraining it. Borrowing the structure of classical feedforward compensation, we identify a *command-response model* of the closed loop formed by the policy and its plant: a network trained on excitation rollouts of the converged policy that predicts, from the current observation and a candidate command, the short-horizon response: body-frame displacement, energy, and terrain-relative posture height. At deployment, *Viability-Gated Feedforward Compensation* (VGFC) inverts this model over a candidate command set: it selects the lowest-predicted-energy command that preserves the direct goal command’s predicted progress, subject to a predictive viability gate that rejects commands whose predicted posture height violates a safety floor, catching height collapse *before* it happens rather than reacting to it. In Isaac Lab rough-terrain experiments on Unitree Go2, VGFC systematically improves the energy economy of the pretrained controller it compensates: across an eight-target benchmark and moderate and held-out target sets it reduces the energy proxy by 5–11% (non-overlapping bootstrap CIs on the main benchmark, $n=400$ per method) while success remains statistically matched, and the paired per-seed energy difference is negative on every one of 14 evaluation seeds (sign test $p < 10^{-4}$). On a second embodiment, the ANYmal-D quadruped, the identical frozen recipe transfers with no re-tuning of any parameter (−3.9% energy at matched success); on the H1 humanoid, whose baseline is already energy-efficient, the uncertainty-aware margin correctly keeps compensation nearly inactive. Component ablations show the design is load-bearing: removing the viability gate forfeits most of the realized saving, unbounding the correction trades success for energy, and a retrieval-based variant (archived behavior chunks reused as gated command residuals) captures only part of the gains and degrades off the archive’s support. Compensation, not replay: a policy’s own experience is most useful as an inverted response model behind a viability gate.

Keywords policy compensation · feedforward control · learned dynamics models · continuous control · legged locomotion · skill archives

1 Introduction

Learning-based continuous control has matured to the point where strong pretrained policies are commodities: massively parallel simulation trains velocity-conditioned locomotion controllers for quadrupeds and humanoids in minutes [Rudin et al., 2022, Makoviychuk et al., 2021], and such policies transfer to challenging real-world terrain [Lee et al., 2020, Miki et al., 2022, Hwangbo et al., 2019]. Yet a converged policy is not a uniformly good policy. Its competence mirrors its training distribution: command tracking is energy-inefficient in regions the curriculum visited rarely; aggressive proportional commands waste effort on terrain where they mostly produce slip; and on long tasks the policy can drift into low-posture regimes it was never penalized for entering. The standard remedies (reward re-

shaping, curriculum redesign, fine-tuning) all require another pass of training, with its cost and its risk of regressing behaviors that already work.

Classical control has a name for the alternative: *feedforward compensation*. When a plant exhibits known systematic error, one identifies a model of its response and uses the inverse of that model to pre-correct the reference signal, before feedback ever has to react [Åström and Murray, 2008, Francis and Wonham, 1976]. This paper develops the analogous construction for learned continuous-control policies. The “plant” is the closed loop formed by the frozen policy and the robot; the “reference” is the high-level command the policy was trained to consume; and the “model” is learned from the policy’s own rollouts, the same self-generated experience that skill-discovery and behavior-archive methods [Eysenbach et al., 2019, Cully et al., 2015, Pertsch et al., 2020] would distill into replayable skills. Where those methods reuse experience by *replaying* it (an approach we show is brittle for contact-rich control), we reuse it by *identifying the system it came from*, then inverting that model at the command interface.

Concretely, we introduce **Viability-Gated Feedforward Compensation (VGFC)**, with three components. (i) *Command-response model*: we excite the converged policy with uniformly sampled velocity commands across many parallel environments, in the manner of system identification with rich excitation, and train a network that maps the current observation and a candidate command to the short-horizon closed-loop response: body-frame displacement and yaw change, per-step energy, and posture height measured *relative to terrain*. (ii) *Feedforward inversion*: at each high-level step, VGFC evaluates a candidate command set spanning speeds, approach angles, and yaw rates, and selects the candidate with the lowest predicted energy among those that retain at least a fixed fraction of the direct goal command’s predicted progress. (iii) *Predictive viability gating*: candidates whose predicted minimum posture height violates a safety floor are rejected *before execution*; if the direct command itself is predicted to violate the floor, VGFC switches to a rescue mode that maximizes predicted posture height; and if measured posture has already sagged, compensation is suspended so the policy’s aggressive tracking can restore it. The learned policy remains the safe default at every branch: compensation degrades to no-op, never to open-loop behavior.

We instantiate VGFC on legged locomotion, the domain where feedback sensitivity makes naive skill reuse fail most sharply, using Isaac Lab rough-terrain target reaching with three embodiments: the Unitree Go2 and ANYmal-D quadrupeds and the H1 humanoid, with the public pretrained RSL-RL checkpoints as the compensated policies and as the baselines. Across *all* evaluated task families (an eight-target Go2 benchmark plus moderate and held-out sets; ANYmal-D on the same benchmark; H1 targets), VGFC reduces energy at statistically matched success, and the paired per-seed energy difference is negative on every Go2 and ANYmal-D evaluation seed; on ANYmal-D the frozen configuration transfers with zero re-tuning. We additionally report component ablations that map the failure modes of naive model inversion (without the viability gate, predicted savings fail to materialize in the closed loop; without the bounded correction, the compensator trades success for energy) and an extensive study of the *retrieval-based* alternative (archived chunks converted to gated command residuals, the natural evolution of skill-archive methods), which captures a smaller energy improvement and degrades off the archive’s support. The gate is in the name because the ablations show it is load-bearing.

Contributions. (1) A *feedforward-compensation framing* for improving fixed learned continuous-control policies: identify the closed-loop command response from self-generated excitation data, then invert it at the command interface at deployment, with no retraining and no action-space intervention (Sections 3.2–3.3). (2) A *predictive viability gate* built on terrain-relative posture prediction, which anticipates height collapse before residual execution; we show terrain-relative labels are essential, raising posture-prediction R^2 from 0.25 to 0.88 over world-frame labels (Section 3.2). (3) A systematic empirical study on Go2, ANYmal-D, and H1 rough terrain, with multiple seeds, tuning/evaluation seed separation, paired per-seed statistics, per-target analysis, and control examples, showing seed-consistent energy reductions at statistically matched success on every task family, including zero-re-tuning transfer to ANYmal-D (Section 4). (4) Component ablations of the compensator itself and a characterization of the *retrieval-based alternative* (viability-gated skill residuals from a quality-diversity-style archive), delineating why compensation needs a model, a constraint, a gate, and a bounded correction, and what each contributes (Sections 4.5, 4.6).

2 Related Work

Feedforward and internal-model control. Feedforward compensation pre-corrects a reference using a plant model, leaving feedback to handle residual error [Åström and Murray, 2008]; the internal model principle formalizes why regulation requires a model of the signals to be tracked or rejected [Francis and Wonham, 1976]. VGFC transports this structure to learned policies: the “plant” is the policy-robot closed loop, the model is a learned network, and inversion is a discrete search over the command set. The novelty is not the control-theoretic template but its instantiation on top of a frozen RL policy, with the model identified from the policy’s own rollouts.

Model-based RL. Learning a dynamics model and exploiting it for control is one of the oldest programs in RL: Gaussian-process models enable data-efficient policy search [Deisenroth and Rasmussen, 2011], probabilistic neural ensembles support sampling-based planning within a handful of trials [Chua et al., 2018], short model-generated rollouts accelerate policy optimization [Janner et al., 2019], latent world models train behaviors entirely in imagination [Hafner et al., 2020], and learned models combine with terminal value functions for planning [Hansen et al., 2022]; on physical robots, neural dynamics models have driven locomotion and navigation directly [Nagabandi et al., 2018]. All of these model the environment at the *action* level so that a policy or planner can be built on top. VGFC inverts the arrangement: the policy already exists and is competent, so what is modeled is the short-horizon response of the *closed loop*, policy included, to its 3-D command input. This collapses the input dimensionality at the interface, makes inversion by enumeration trivial, and confines the search space to “commands the policy understands” rather than raw torques.

MPC in robotics and vehicles. Model predictive control optimizes a model-based cost over a receding horizon under constraints [Mayne et al., 2000], and is a workhorse of both vehicle and legged control: predictive steering on slippery roads [Falcone et al., 2007], sampling-based path-integral control for aggressive autonomous driving [Williams et al., 2018], learned-dynamics MPC for autonomous racing [Kabzan et al., 2019], and convex MPC for quadruped locomotion [Di Carlo et al., 2018]; learning-based MPC with safety guarantees is a field of its own [Hewing et al., 2020]. VGFC can be read as the smallest useful member of this family: a sampling-based, single-interval MPC whose “dynamics” are an identified closed-loop response, whose decision variable is one velocity command rather than an action sequence, and whose constraint handling is a veto gate with a guaranteed fallback instead of constrained optimization. The simplification is deliberate: the low-level policy already solves the hard receding-horizon problem at the contact timescale, so the compensator only needs to choose *which* well-understood command to issue next.

Reference governors and safety filters. The structure of VGFC has a precise control-theoretic ancestor: the reference governor, an add-on scheme that monitors and minimally modifies the reference of a pre-stabilized closed loop to enforce constraints [Garone et al., 2017], and its learning-era descendant, the predictive safety filter, which certifies or corrects a proposed input by model prediction [Wabersich and Zeilinger, 2021]. VGFC can be read as a *learned, cost-optimizing* governor: where governors modify the reference only as much as constraint enforcement demands, VGFC additionally optimizes a running cost (energy) at matched task progress, and its “model” is identified from rollouts rather than derived, trading the governors’ invariance guarantees for applicability to policies whose dynamics exist only as a network. Two recent robot-learning systems are the nearest neighbors. ABS [He et al., 2024] searches twist commands against a learned reach-avoid value to keep an agile quadruped policy collision-free; its objective is safety, and its value network and policies are co-trained as one framework, whereas VGFC attaches to an unmodified, off-the-shelf policy and optimizes efficiency subject to safety. Kim et al. [2024] identify linear Koopman dynamics of an RL-controlled biped, policy in the loop, for MPC-based safe navigation; VGFC likewise identifies the closed loop at the command level, but with an observation-conditioned nonlinear model whose terrain-dependent energy and posture channels have no linear lifted-state analogue, and uses it for compensation rather than navigation. To our knowledge, the combination VGFC instantiates—post-hoc identification of a frozen policy’s situational command response, running-cost optimization at matched predicted progress, and a learned predictive viability gate behind a bounded correction—has not been reported, though each mechanism, as this section shows, has clear ancestors.

Options, skills, and behavior archives. Temporal-abstraction research builds libraries of reusable behaviors: options [Sutton et al., 1999, Bacon et al., 2017], unsupervised skills [Eysenbach et al., 2019, Sharma et al., 2020], skill priors from offline data [Pertsch et al., 2020, Ajay et al., 2021], large-scale skill embeddings [Peng et al., 2022], and quality-diversity repertoires used for rapid adaptation [Mouret and Clune, 2015, Cully et al., 2015, Pugh et al., 2016]. DADS [Sharma et al., 2020] is notable here for planning over learned skill-transition models, but those models are co-trained with the skills they describe; VGFC instead identifies the response of an already-trained task policy it had no part in shaping. Our retrieval-based variant (Section 3.4) is squarely in this tradition: an online archive of behavior chunks with body-frame outcome descriptors, reused as gated command residuals. Our experiments position it as a partial solution: it inherits the archive’s coverage limits, whereas the response model interpolates over the whole command box. Hierarchical RL layers goal-conditioned controllers [Nachum et al., 2018]; recent imitation methods execute action chunks closed-loop [Zhao et al., 2023]. We show raw chunk replay destabilizes contact-rich locomotion, motivating compensation at the command interface instead.

Improving a pretrained policy without retraining it. Residual policy learning adds a learned action-space correction on top of a base controller [Silver et al., 2018, Johannink et al., 2019]; it improves the base but requires reward access and further environment interaction, and its correction shares authority at the contact-critical timescale. RMA adapts a policy online through a latent extrinsics estimate [Kumar et al., 2021], and multiplicity-of-behavior training exposes a space of gaits through the command interface [Margolis and Agrawal, 2022]; both must be built in at training time. Massively parallel pipelines [Rudin et al., 2022, Makoviychuk et al., 2021, Mittal et al., 2023, Schwarke et al., 2025] and terrain curricula [Lee et al., 2020, Miki et al., 2022] improve the policy during training. VGFC occupies the

remaining cell of this matrix: it is attached after training, learns nothing by reinforcement, touches only the command interface the policy already exposes, and computes its correction by inverting an identified model, so the worst case of a poor correction is inefficiency rather than a fall.

3 Method

Design requirements. Three requirements shape every design decision that follows. *R1, no retraining:* the compensator must improve a policy whose weights, reward, and training pipeline are unavailable or too expensive to touch; it may only observe the policy acting. *R2, do no harm:* the pretrained policy is strong, so the compensator must be able to fall back to it exactly, and every intervention must be vetoed when its predicted benefit is uncertain. *R3, embodiment generality:* nothing in the method may depend on robot-specific knowledge beyond what can be identified automatically from the policy’s own rollouts. The resulting architecture has three parts, presented in the next three subsections: a compensation *interface* (where to intervene, Section 3.1), a response *model* (what to know, Section 3.2), and a gated *inversion* rule (how to act, Section 3.3).

3.1 Compensation Interface

Let π_θ be a frozen, pretrained continuous-control policy mapping observations o_t and a command c to low-level actions. In our locomotion instantiation $c = (v_x, v_y, \omega_z)$ is the body-frame velocity command, but the framework only requires *some* command interface through which behavior can be steered, a property shared by learned controllers that are deployed as task-level building blocks [Margolis and Agrawal, 2022, Kumar et al., 2021].

Scope. Three properties of the policy, not of robots, delimit where the framework applies. (i) *A command interface must exist:* a single-task policy trained without conditioning exposes nothing to invert over, so VGFC does not apply to unconditioned RL policies (a limitation shared by every command-level method). (ii) *The interface must carry redundancy:* compensation exploits slack, i.e., several commands making comparable task progress at different cost; a task that pins the command leaves nothing to save. (iii) *The closed loop must respond at a fast, roughly stationary timescale:* command consequences must be visible within a short prediction window, which holds when the policy stabilizes its plant (locomotion, end-effector servoing, vehicle velocity/curvature tracking) and fails for sparse, long-horizon credit-assignment problems. These conditions characterize command-conditioned continuous control broadly, from legged robots to vehicle controllers, rather than reinforcement learning in general; Section 5 returns to what lies outside them.

Why intervene at the command rather than at the action, as residual policy learning does [Silver et al., 2018, Johannink et al., 2019]? Three reasons, mirroring the three requirements. First, action-space corrections must be *learned*, and learning them requires reward access and environment interaction, violating R1; a command-space correction can instead be *computed* from an identified model. Second, an action-space residual shares authority with the policy at the contact-critical timescale, where a wrong correction destabilizes; a command-space correction leaves π_θ in full control of posture, contacts, and joints, so the worst case of a poor command is inefficiency, not a fall (R2). Third, the command box is the one interface that command-conditioned policies share across embodiments (R3): the same three numbers steer a quadruped and a humanoid.

A task supplies a target g ; the *direct* controller computes a proportional body-frame command

$$c_{\text{goal}} = (\text{clip}(k_v x_{\text{local}}), \text{clip}(k_v y_{\text{local}}), \text{clip}(k_\psi \text{atan2}(y_{\text{local}}, x_{\text{local}}))), \quad (1)$$

which is the uncompensated baseline: a strong pretrained policy driven straight at the goal. Proportional commands of this kind are the standard way tasks are layered on velocity-conditioned policies, and they carry a systematic inefficiency: far from the goal they saturate, commanding full speed regardless of whether the terrain rewards it, and near the goal they shrink, producing energy-hungry in-place stepping. Compensation replaces c_{goal} with a corrected command at each high-level step (every $E = 4$ low-level steps). The design question is where the correction comes from and when it is trusted.

3.2 Command-Response Model

To correct a command before executing it, the compensator must predict what any candidate command would do. The object to model is not the robot’s action-level dynamics, as in model-based RL [Nagabandi et al., 2018]: the policy already handles those competently, and re-modeling them would duplicate its work at far higher dimensionality. What needs modeling is the *closed loop* of policy and robot, viewed from the command interface: given the current situation, what motion, at what energetic price, and at what postural risk does each command buy? This reframing keeps the

model input three-dimensional at the interface and makes identification a supervised problem on the policy’s own rollouts, satisfying R1 and R3 simultaneously.

Excitation data. We run the converged policy in 64 parallel environments and override its command with excitation commands sampled uniformly from the command box (scaled down with probability 0.3 to cover the near-goal regime), resampled every 25 steps. This is system identification with persistent excitation, applied to the policy-robot closed loop. Crucially, *no quality filtering is applied*: stumbles, slips, and low-posture episodes are kept, because the model must know where the policy performs poorly. This is the same self-distillation data source that populates our behavior archive (Section 3.4); the difference is what is extracted from it.

Response targets. The outputs instantiate a general three-channel schema that any instantiation of the framework must fill: a *progress* statistic sufficient to score task advancement, the *cost* being minimized, and the *viability* quantity the gate guards. For locomotion, from constant-command windows of $H=16$ low-level steps we regress six outcomes: body-frame displacement $(\Delta x, \Delta y)$ and yaw change $\Delta\psi$ (progress), per-step energy $\bar{e} = \text{mean}(|a_t \odot \dot{q}_t|)$ (cost), and the minimum and mean of *terrain-relative posture height* over the window (viability); a manipulation instantiation would substitute end-effector displacement, actuation effort, and a grasp-stability or joint-limit margin, and a vehicle instantiation path progress, consumption, and a slip margin. The horizon is four replanning intervals ($E=4$ low-level steps per high-level decision, 80 ms at the 50 Hz control rate): long enough to span a stride and expose gait-level energy and posture differences between commands, short enough that a single-step prediction from the current observation remains accurate. Posture height is recovered from the policy’s own height-scan observation (mean scan return plus sensor offset), not from world coordinates. This choice matters: on procedurally rough terrain, world-frame base height confounds posture with ground elevation, and a model trained on world-frame labels reaches only $R^2 = 0.25$ on minimum height; the terrain-relative labels raise this to $R^2 = 0.88$ (Table 1). An unreliable height channel silently disables predictive gating; this label correction is what makes the viability gate work.

Model. The response model is a mapping

$$f_\phi : (o_t, c) \mapsto (\widehat{\Delta x}, \widehat{\Delta y}, \widehat{\Delta\psi}, \hat{e}, \hat{h}_{\min}, \hat{h}_{\text{mean}}), \quad (2)$$

implemented as a two-layer MLP (256 units, dropout 0.1) that takes the full policy observation with the command slice replaced by the candidate c ; inputs and outputs are standardized, and training minimizes MSE with an episode-grouped train/test split so windows from one episode never straddle the split. Conditioning on the full observation is what makes the predictions *situational*: the same candidate command is predicted to cost different energy on different terrain and from different gait states, which is precisely the information a fixed command-shaping heuristic cannot express. Identification is cheap: $\sim 25\text{k}$ windows from roughly four minutes of parallel simulation, and minutes of training.

3.3 Viability-Gated Inversion

Given the response model, the naive compensator would pick the command whose predicted outcome looks best. The pitfalls are instructive, and each safeguard below exists because we hit one of them (quantified in Section 4.6). Minimizing predicted distance-to-goal turns the compensator into a speed maximizer, since aggressive commands always look best over a short horizon; so *progress is a constraint, not the objective*. Minimizing energy without posture safeguards finds the policy’s true energetic optimum, a low crawl that fails the task; so *viability gates the objective*. And executing the optimizer’s choice verbatim concentrates all model error into the executed command; so *the correction is bounded*.

At each high-level step VGFC forms a candidate set $\mathcal{C}$ around the direct command: the direct command itself, scaled versions, yaw-rate perturbations, and a polar grid over approach speeds and angles with three yaw-rate levels (~ 70 candidates after deduplication). A single batched forward pass predicts each candidate’s response $(\widehat{\Delta x}, \widehat{\Delta y}, \widehat{\Delta\psi}, \hat{e}, \hat{h}_{\min}, \hat{h}_{\text{mean}})$, from which predicted progress is

$$\hat{P}(c) = \|g_{\text{body}}\| - \|g_{\text{body}} - (\widehat{\Delta x}(c), \widehat{\Delta y}(c))\|. \quad (3)$$

Selection then proceeds in one of three modes (Algorithm 1):

(a) *Follow (energy compensation)*. If the direct command is predicted posture-safe, VGFC selects

$$c^* = \arg \min_{c \in \mathcal{C}_{\text{elig}}} \hat{e}(c), \quad \mathcal{C}_{\text{elig}} = \left\{ c : \hat{P}(c) \geq \beta \hat{P}(c_{\text{goal}}), \hat{h}_{\min}(c) \geq h_{\text{floor}}, \hat{h}_{\text{mean}}(c) \geq h_{\text{now}} - \delta \right\}, \quad (4)$$

with progress floor $\beta = 0.9$, posture floor h_{floor} a fixed fraction of nominal posture height, and height-decline tolerance $\delta = 2 \text{ cm}$. The compensation activates only if the predicted saving clears a margin, $\hat{e}(c^*) < \hat{e}(c_{\text{goal}}) - \epsilon_e$; otherwise the

Algorithm 1 Viability-Gated Feedforward Compensation (one high-level step)

Require: frozen policy π_θ , response model f_ϕ , target g , scalars $(\beta, h_{\text{floor}}, \delta, \epsilon_e, \alpha_0, d_{\text{anneal}})$

- 1: Compute body-frame target g_{body} and direct command c_{goal} (Eq. 1)
- 2: Build candidate set $\mathcal{C}$; predict responses $f_\phi(o, c)$ for all $c \in \mathcal{C}$ in one batch
- 3: **if** measured posture $h_{\text{now}} < \text{operating band}$ **then** ▷ restore
- 4: $c^* \leftarrow c_{\text{goal}}$
- 5: **else if** $\hat{h}_{\min}(c_{\text{goal}}) < h_{\text{floor}}$ **then** ▷ rescue: predicted collapse
- 6: $c^* \leftarrow \arg \max_{c: \hat{P}(c) \geq 0} \hat{h}_{\min}(c)$
- 7: **else** ▷ follow: energy compensation
- 8: $c^* \leftarrow \arg \min_{c \in \mathcal{C}_{\text{elig}}} \hat{e}(c)$ per Eq. 4
- 9: **if** $\hat{e}(c^*) \geq \hat{e}(c_{\text{goal}}) - \epsilon_e$ **then** $c^* \leftarrow c_{\text{goal}}$
- 10: **end if**
- 11: **end if**
- 12: Execute π_θ with $c = c_{\text{goal}} + \alpha_t(c^* - c_{\text{goal}})$ for E low-level steps ▷ bounded, annealed; Eq. 5

direct command runs unmodified. The semantics are exactly feedforward compensation: *keep the reference trajectory, remove the systematic energetic inefficiency.*

(b) *Rescue (predictive viability).* If the *direct command itself* is predicted to drive minimum posture height below a rescue threshold (set below the eligibility floor, so ordinary roughness does not trigger it), meaning the model foresees height collapse, VGFC instead selects the candidate maximizing predicted minimum posture height among those with non-negative predicted progress. This is the gate acting before failure exists: reactive height gates triggered by *measured* height, which we tested in the retrieval variant, either block progress or fire after posture is already lost.

(c) *Restore (measured posture).* If measured posture height has already sagged below an operating band, compensation is suspended and the direct command runs until posture recovers; the aggressive proportional command restores height reliably. This closes the loop against slow posture drift that a 16-step model horizon cannot see.

Bounded, annealed correction. In all modes the selected candidate is applied as a *bounded feedforward correction* rather than a substitute, and the correction gain decays as the goal approaches:

$$c = c_{\text{goal}} + \alpha_t (c^* - c_{\text{goal}}), \quad \alpha_t = \alpha_0 \text{clip}(\|g_{\text{body}}\|/d_{\text{anneal}}, 0, 1), \quad \alpha_0 = 0.5. \quad (5)$$

Bounding ($\alpha_0 < 1$) means the compensator biases the reference toward the model’s preferred command but never fully owns it; this is the classical feedforward posture (correct the reference, keep the controller), and Section 4.6 shows it is what makes the method robust to model error: full substitution matches the bounded correction on the tuning seed and loses 5–6 success points off it. Annealing ($d_{\text{anneal}} = 0.5$ m) returns full authority to the base policy for the terminal approach, because the success criterion is evaluated at the terminal state, which lies beyond the model’s prediction horizon for most of the episode. The three modes are checked in the order restore, rescue, follow, so measured evidence overrides predicted evidence. The compensator itself is essentially free at run time: one batched forward pass over all candidates costs 0.14 ms on CPU (0.08 ms on GPU) against the 80 ms high-level control interval.

Altogether VGFC exposes seven scalars: the progress floor $\beta = 0.9$, the eligibility and rescue posture floors (0.85 and 0.80 of nominal height), the restore band (0.90), the relative energy margin (10%), and the correction gain and annealing distance of Eq. 5. All were tuned once, on a single seed of one robot, and frozen for every robot and task family we evaluate (Section 4.1).

3.4 Retrieval Variant (VGSR)

Before arriving at model inversion, we developed *Viability-Gated Skill Residuals* (VGSR), which we report in full as the natural skill-archive alternative. The same converged-policy rollouts are segmented into chunks, described by body-frame outcome descriptors (displacement, velocities, yaw rate, height statistics, energy, stability, smoothness), filtered by utility and descriptor novelty, and clustered into an online quality-diversity-style archive [Cully et al., 2015]. At deployment, a chunk is selected by a descriptor-level cost combining predicted progress, initial-state applicability, energy, stability, and utility; its velocity descriptor is converted into a command that is *blended* with the direct command, $c = (1 - \alpha)c_{\text{goal}} + \alpha c_{\text{skill}}$, with $\alpha > 0$ only when a heuristic viability gate (goal alignment, predicted progress, chunk stability, energy z-score, speed cap) accepts the candidate; a learned decision-level gate can further filter it. VGSR is the non-parametric cousin of VGFC: retrieval interpolates only where the archive has coverage, while the response model interpolates over the entire command box; and VGSR’s gate checks properties *of an archived chunk*,

while VGFC’s gate checks *predicted consequences of the actual candidate command in the current state*. Section 4.5 quantifies the difference.

4 Experiments

Our experiments answer four questions. *Q1*: does compensation improve a strong pretrained policy, and on which metrics (Sections 4.3, 4.4)? *Q2*: does the recipe transfer across embodiments without re-tuning (Section 4.4)? *Q3*: which components carry the effect (Section 4.6)? *Q4*: how does model inversion compare with the retrieval-based use of the same experience (Section 4.5)?

4.1 Setup

Environments and robots. All experiments use Isaac Lab [Mittal et al., 2023] rough-terrain velocity environments with three embodiments: Isaac-Velocity-Rough-Unitree-Go2-v0 and Isaac-Velocity-Rough-Anymal-D-v0 (quadrupeds) and Isaac-Velocity-Rough-H1-v0 (humanoid), with the publicly distributed pretrained RSL-RL checkpoints [Rudin et al., 2022, Schwarke et al., 2025] as the compensated policies. The task is target reaching: move the base to a target g specified relative to the start pose. Success requires no early termination, final XY distance below 0.3 m, and *final height* above 70% of spawn height. Final height is the world-frame height of the base at the episode’s last step; it serves as a terminal posture check, so that an episode that reaches the target crouched, collapsed, or mid-fall does not count as a success. Because the terrain is procedurally rough, final height reflects both posture and the ground elevation at wherever the robot stops, which is why we report it alongside, rather than inside, the energy and distance metrics. We also report final distance, the energy proxy $\text{mean}(|a \odot \dot{q}|)$ (mean absolute mechanical power over the episode), and how often compensation was active.

Task families. *Harder* Go2 targets: (2, 0), (1.5, 1), (0, 1.25), (−0.75, 0), (1, −1), (0, −1.25), (−1, 0.75), (1.75, −0.75) m: long, diagonal, lateral, and backward goals covering the command box (the last three directions were added when the benchmark was expanded and were never used in any tuning). *Moderate* Go2 targets: (0.75, 0), (0.75, 0.75), (1.5, 0), (0, 0.75) m. *Holdout* Go2 directions (never used during any tuning): (1.25, 0.5), (0.5, 1), (−0.5, 0.5), (1.25, −0.5) m. H1 targets: (0.5, 0), (0.5, 0.5), (1, 0) m. ANYmal-D uses the same eight harder targets as Go2, with the entire recipe (excitation collection, identification, and every gate parameter) applied unchanged: no per-robot tuning of any kind.

Methods. (1) *direct*: the pretrained policy under the direct target command (the policy being compensated, and a strong baseline); (2) *VGSR*: the retrieval-based variant with heuristic gate (and recovery clause on harder targets); (3) *VGFC*: model-feedforward compensation (Section 3.3). Random/zero-command baselines are excluded as uninformative against a strong pretrained policy.

Protocol. VGFC’s seven scalars (Section 3.3) were tuned on a *single* seed (301) on the harder set only, then frozen. All reported results use disjoint evaluation seeds: harder 5 seeds $\times$ 10 trials $\times$ 8 targets (400 episodes per method), moderate 3 $\times$ 10 (120), holdout 3 $\times$ 5 (60), ANYmal-D 3 $\times$ 5 $\times$ 8 (120), H1 3 $\times$ 5 (45). Per-robot response models (Go2, ANYmal-D, H1) are identified with the identical recipe. We report episode-level bootstrap 95% CIs.

4.2 Response-Model Quality

Table 1 reports held-out-episode test metrics for the Go2 response model. Displacement and yaw responses are accurately predicted ($R^2 \geq 0.92$); energy is predicted well enough to rank candidates ($R^2 = 0.75$); and terrain-relative posture height is predicted reliably ($R^2 = 0.88/0.91$ for min/mean, MAE under 9 mm), versus $R^2 = 0.25$ when the same model is trained on world-frame height labels, which terrain elevation corrupts. The excitation data also matters: an earlier model trained on utility-filtered archive segments (the natural “reuse the archive” shortcut) reached $R^2 = -1.9$ on lateral displacement, because curated high-quality segments underrepresent exactly the sloppy behavior the model must know about.

Identification data efficiency. Re-identifying the model from episode-group subsamples of the excitation data degrades gracefully: with 50% of the data, displacement R^2 holds at 0.93 and energy drops to 0.72; with 25% ($\approx$ one minute of parallel simulation), displacement is still 0.92 and energy 0.62. Displacement, which drives the progress constraint, saturates early; energy, the objective being optimized, is what additional data buys. Section 4.6 evaluates the 25% model in closed loop.

Table 1: Go2 command-response model, held-out-episode test metrics ($\sim 25k$ excitation windows, 16-step horizon). Terrain-relative height labels are essential for the viability gate.

Output	MAE	RMSE	R^2
Δx (m)	0.019	0.025	0.944
Δy (m)	0.019	0.025	0.942
$\Delta \psi$ (rad)	0.029	0.042	0.927
energy (per step)	0.435	0.625	0.752
min. rel. height (m)	0.009	0.012	0.875
mean rel. height (m)	0.008	0.010	0.906
min. height, world-frame labels	0.039	0.092	0.247

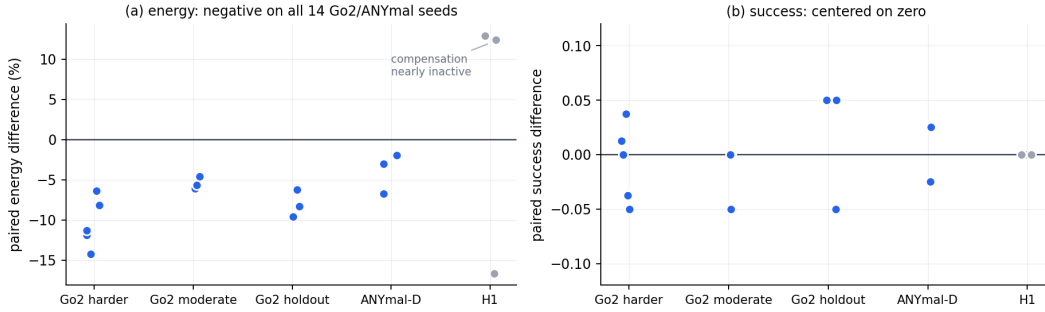


Figure 1: Paired per-seed differences (VGFC – direct, evaluated inside the same runs) for every evaluation seed of every task family. Left: the relative energy difference is negative on all 14 Go2 and ANYmal-D seeds (sign test $p < 10^{-4}$); the H1 seeds (gray) scatter widely because compensation is nearly inactive there. Right: the success difference is centered on zero. The compensation buys energy without paying success.

4.3 Go2 Harder Targets: Main Result

The harder-target benchmark comprises eight targets (five original plus three added directions, Section 4.1) evaluated over 5 seeds $\times$ 10 trials, i.e. 400 episodes per method.

Aggregate. VGFC reduces energy by 10.5% (1.976 vs. 2.208; CIs [1.938, 2.016] vs. [2.156, 2.256], non-overlapping) while success is statistically matched (0.797, CI [0.760, 0.835], vs. 0.805, CI [0.765, 0.843]), with matched final distance and height and zero early terminations for either method. The energy effect is consistent *within* seeds: the paired per-seed difference is negative on all five benchmark seeds (-6.4% to -14.3%), and on all 14 evaluation seeds across the Go2 families and ANYmal-D (sign test $p < 10^{-4}$), while the paired success difference is centered on zero (-0.05 to $+0.04$). Figure 1 visualizes both distributions; we summarize the aggregate in text because the honest headline is one number and one non-effect, not a grid of near-identical bars. The retrieval variant, evaluated on the original five-target protocol (250 episodes), reaches a similar per-family energy level (2.061 vs. VGFC’s 2.073 under that protocol) but with a 1.2% termination rate and lower final height, and, as Section 4.5 shows, its savings do not persist across the other task families.

Control examples. Figure 2 overlays the closed-loop XY paths for all trials of one evaluation seed: compensated and uncompensated trajectories are nearly indistinguishable in space, which is the intent, since the compensation acts on command tracking rather than on the route. Figure 3 dissects a single episode on the hardest target and shows the predictive rescue mechanism at work: the model forecasts a posture-floor breach early in the walk, rescue-mode candidates hold the base height through the risky stretch, and the episode ends at nominal posture while the direct baseline sinks below the success threshold.

Per-target (Table 2). The per-target view explains the aggregate. Energy improves on seven of eight targets (up to -14%), with the one exception, the short backward goal, at parity; final height is within a few millimeters everywhere. Success differences scatter in both directions at the per-target sample size ($n=50$): the long diagonal (1.5, 1) now favors VGFC (0.800 vs. 0.740, the target the annealed correction was designed to protect), the two long forward/diagonal goals favor direct by a similar margin, and the remaining five are matched. We refrain from claiming any single-target success effect: across protocol versions these $n=50$ differences moved by ± 0.1 while the energy ordering never changed, which is exactly what a real energy effect and a null success effect look like.

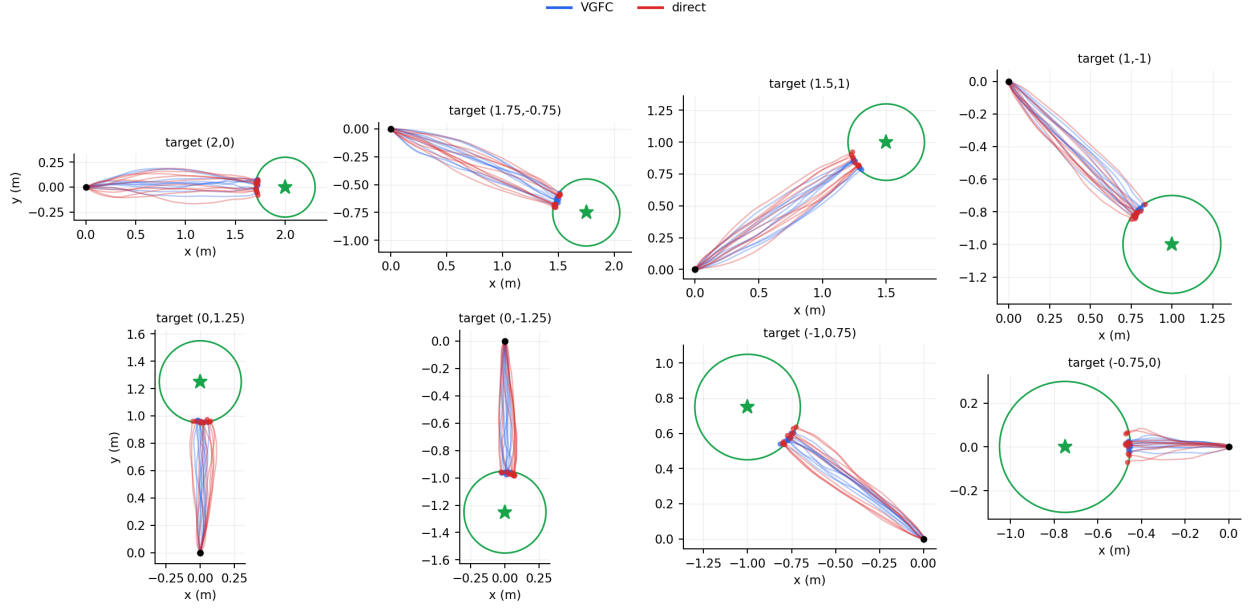


Figure 2: Control examples: XY paths on the eight harder targets (one evaluation seed, 10 trials per method; the green circle is the success radius). VGFC and the direct baseline reach the same target neighborhoods along closely matched paths; the compensation changes how the command is tracked, not where the robot goes.

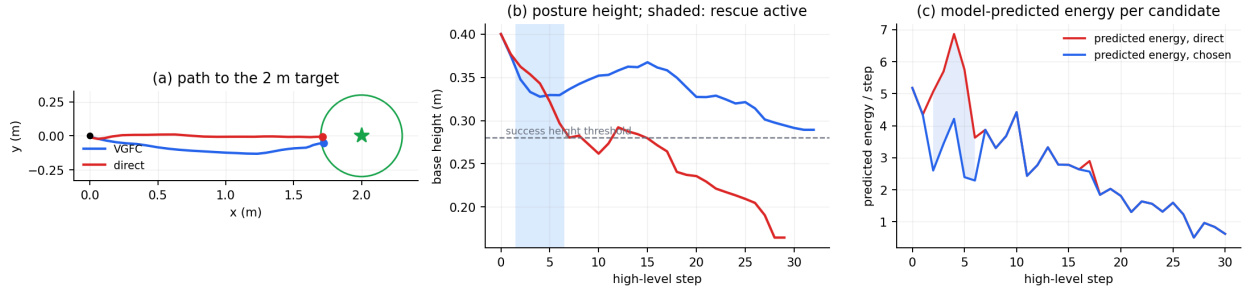


Figure 3: A single compensated episode on the hardest target (2, 0). (a) Both controllers walk 2 m over rough terrain. (b) Base height over the episode: during the shaded steps the model predicts that the direct command would breach the posture floor, and the rescue mode substitutes height-preserving candidates; VGFC holds posture while the uncompensated policy sinks below the success threshold. (c) The response model’s per-step energy predictions for the direct command and the chosen candidate; the shaded gap is the predicted saving that the margin requires before compensating.

On final height. Compensated final height sits 1–3 mm below the baseline on the Go2 families (0.3228 vs. 0.3253 here, with near-identical CIs). The mechanism is inherent to the objective: among posture-safe candidates the energy minimizer prefers marginally gentler commands, and on a velocity-conditioned policy gait vigor and carried height are weakly coupled, so the cheapest safe gait stands fractionally lower. The posture floors and the restore mode bound this coupling to the millimeter scale (early unconstrained variants lost centimeters, and the task, with them; Section 4.6), and the effect is not intrinsic to compensation: on ANYmal-D and H1 the compensated policy stands *taller* than the baseline (Table 3).

On the residual failure mode. Every failure on this benchmark, for both methods (81 and 78 of 400 episodes respectively), is of a single kind: the robot reaches the target region but finishes below the height criterion after its gait has settled low over the walk. We tested whether a posture-aware stopping rule could convert these: under a symmetric protocol where *neither* controller may declare the task done while posture is low (the direct baseline keeps tracking;

Table 2: Per-target breakdown on the eight harder targets (50 episodes per target per method), ordered by the paired energy saving. Energy improves on seven of eight targets, by up to 12.9%; success differences at this sample size scatter in both directions around parity, and final height is matched to a few millimeters.

Target (m)	Energy ↓			Success ↑		Final height (m)	
	VGFC	direct	Δ	VGFC	direct	VGFC	direct
(0, 1.25)	1.815	2.083	−12.9%	0.940	0.980	0.340	0.351
(1.75, −0.75)	2.217	2.542	−12.8%	0.480	0.520	0.276	0.285
(−1, 0.75)	1.861	2.122	−12.3%	1.000	1.000	0.358	0.367
(0, −1.25)	1.822	2.073	−12.1%	0.940	0.980	0.345	0.350
(1.5, 1)	2.203	2.471	−10.8%	0.800	0.740	0.316	0.313
(2, 0)	2.367	2.619	−9.6%	0.240	0.260	0.233	0.224
(1, −1)	2.028	2.240	−9.5%	0.980	0.960	0.355	0.349
(−0.75, 0)	1.496	1.510	−0.9%	1.000	1.000	0.359	0.363

Table 3: Systematic coverage with one frozen configuration per robot. Energy Δ is VGFC relative to the direct baseline evaluated inside the same runs. Success stays statistically matched everywhere while energy drops on all Go2 families and ANYmal-D; on H1, whose baseline is already efficient, the uncertainty-aware margin keeps compensation nearly inactive (2.0 activations/episode) and the energy difference is seed noise.

Family (n /method)	Method	Success ↑	Energy ↓	Energy Δ	Final height (m)
Go2 moderate (120)	VGFC	0.975 [0.942, 1.000]	1.743 [1.689, 1.802]	−5.4%	0.356
	direct	0.992 [0.975, 1.000]	1.844 [1.763, 1.932]	—	0.362
Go2 holdout (60)	VGFC	0.933 [0.867, 0.983]	1.936 [1.836, 2.037]	−8.0%	0.334
	direct	0.917 [0.833, 0.983]	2.105 [1.982, 2.234]	—	0.333
ANYmal-D, 8 targets (120)	VGFC	0.883 [0.825, 0.942]	1.367 [1.317, 1.422]	−3.9%	0.500
	direct	0.875 [0.817, 0.933]	1.422 [1.373, 1.471]	—	0.500
H1 (45)	VGFC	1.000	0.549 [0.496, 0.605]	+2.1%	1.019
	direct	1.000	0.538 [0.496, 0.582]	—	1.011

VGFC selects the candidate with the highest predicted posture among those staying in the target region), success does not improve for either method at $n=400$ per method; episodes instead spend their remaining budget attempting recovery, occasionally falling (terminations appear for both), and the extra low-motion steps dilute the per-step energy metric. Individual episodes are recoverable, as Figure 3 shows, but at scale the arrive-low state is dominated by terrain and gait-settling dynamics that command-level intervention does not reliably reverse. This bounds what any compensator at this interface can claim on this success criterion, and is why our headline is energy at matched success rather than success itself.

4.4 Systematic Coverage: Moderate, Holdout, and H1

A compensation layer must help where the base policy is weak, do no harm where it is strong, and transfer across embodiments without hand-tuning. Table 3 evaluates the same frozen configuration on the remaining task families and on two further robots.

Moderate and holdout targets. Energy drops on both families (−5.4% and −8.0%) at statistically matched success (0.975 vs. 0.992 and 0.933 vs. 0.917, CIs overlapping in both directions). The holdout family matters most: these directions were never seen during any tuning, so the frozen gates generalize across the command box, which is the property retrieval cannot offer (Section 4.5: the archive variant *costs* energy on these directions).

Third embodiment, zero re-tuning: ANYmal-D. Applying the recipe unchanged to a different quadruped on the full eight-target benchmark reduces energy by 3.9% at matched success (0.883 vs. 0.875), with identical final posture and zero terminations; the paired energy difference is negative on all three ANYmal seeds. Since not a single parameter was adjusted, this is evidence that the compensation exploits a structural property of proportional command tracking on rough terrain rather than a Go2-specific quirk, and that the identification recipe is the only per-robot component the method needs. (The retrieval variant is absent from this row by construction: VGSR requires building and curating a per-robot behavior archive, which is exactly the dependency the response model removes.)

Table 4: VGFC component ablations on the eight-target benchmark (2 seeds $\times$ 5 trials $\times$ 8 targets = 80 episodes per variant). Δ columns are paired against the direct baseline evaluated inside the same runs; at $n=80$ success differences below ~ 0.08 are noise-level, so the full-protocol estimates of Section 4.3 remain the calibrated numbers.

Variant	Success	Δ succ (paired)	Energy	Δ energy (paired)	Comp. steps
VGFC (full)	0.812	+0.012	1.987	−12.2%	11.7
no annealing	0.800	+0.013	2.014	−12.2%	12.3
no viability gate	0.800	−0.025	2.142	−4.8%	8.9
no progress floor	0.787	−0.037	1.783	−19.6%	19.8
substitution ($\alpha=1$)	0.750	−0.050	1.890	−14.0%	16.3
25% identification data	0.800	−0.012	2.077	−6.4%	10.7

H1 humanoid. The H1 baseline walks at ≈ 0.52 energy per step, already near its efficiency floor, and the H1 response model’s energy resolution is coarse relative to that scale (MAE 0.21, $\approx 40\%$ of the per-step baseline, versus 18% for Go2). With the margin sized to that uncertainty (25% required predicted saving), VGFC activates only 2.0 times per episode, and the result is the correct one: saturated success, slightly taller posture, and an energy difference that is pure seed noise (+2.1% mean, with per-seed values swinging from -17% to $+13\%$ at 15 episodes per seed; Figure 1). The gate declining to compensate an already-efficient policy is the intended behavior of an uncertainty-aware margin, and it is exactly what the absolute-margin ablation of Section 4.6 failed to do.

4.5 Retrieval-Based Compensation: What the Archive Buys

The retrieval variant was evaluated extensively before VGFC existed; we first compare head-to-head, then summarize the study it produced.

Head-to-head. The three methods were evaluated inside the same runs on the original five-target protocol (5 seeds $\times$ 10 trials, 250 episodes per method, pre-annealing VGFC configuration). There, VGSR reaches VGFC’s energy level on the harder family (2.061 vs. 2.073, both $\approx 12\%$ below direct) but with a 1.2% termination rate and lower final height; on moderate targets its saving is smaller at lower success; and on holdout directions and H1 it *costs* energy (+3.8%, +4.4%) where VGFC saves (−8%) or stays neutral. The pattern is exactly the coverage argument of Section 3.4: retrieval compensates well where the archive is dense (the harder targets it was built around) and degrades off its support, while the response model generalizes across the command box.

Multi-seed aggregate. In a five-seed, 200-episode-per-method aggregate on the harder targets, VGSR reduced energy by 7.8% (1.99 vs. 2.16, non-overlapping bootstrap CIs) at statistically indistinguishable success (0.785 vs. 0.800); consistent, but smaller than VGFC’s gains and without success improvement.

Gate necessity. Removing VGSR’s gate leaves final distance intact but inflates energy by over 50% (3.18 vs. 2.08), with the residual firing at nearly twice the gated rate: the archive without an activation mechanism is a net energetic negative.

Archive capacity is not the knob. Trimming the 3,904-chunk archive to its top-2,000 chunks by utility *improved* success on a shared seed (0.840 vs. 0.680), while a top-500 archive gave the lowest energy (1.75) at reduced success (0.760); all sizes reduced energy versus direct. Quality filtering, not capacity, carries the retrieval variant, and its non-monotonicity is a tuning liability the response model does not have.

Learned decision gates. A decision-level viability classifier trained on logged gate decisions (episode-group split, heuristic-gate outputs excluded from inputs) reaches AUC 0.97, and deployed in intersection with the physical guard it is stable on held-out directions; but trained on episode-level success labels it learns “low-energy, goal-approaching” as a proxy and loses success on the height-limited 2 m target. VGFC’s gate resolves exactly this: its safety signal is a *predicted physical quantity* (posture height) rather than a correlate of episode outcome.

4.6 Ablating VGFC’s Components

Which parts of VGFC carry the result? We ablate one component at a time on the eight-target benchmark under a shared protocol (2 seeds $\times$ 5 trials $\times$ 8 targets = 80 episodes per variant, with the paired direct baseline inside each run; Table 4): *no annealing* disables the goal-proximity decay of the correction gain; *no gate* removes all posture safeguards (height floors, rescue, and restore), leaving only the progress floor; *no progress floor* sets $\beta = 0$, leaving only the posture safeguards; *substitution* sets $\alpha = 1$, executing the selected candidate verbatim; and *25% data* uses a response model identified from a quarter of the excitation data ($\approx$ one minute of simulation).

The ablations separate the components’ roles. **The gate converts predicted savings into realized ones.** Removing the posture safeguards does not unleash deeper savings; it forfeits most of them (-4.8% vs. -12.2%) while flipping the paired success difference negative: candidates that look efficient to the model but leave the well-behaved posture regime burn their predicted savings in the recovery dynamics that follow. The viability gate is not a tax on compensation; it is the mechanism by which open-loop predictions survive contact with the closed loop. **The progress floor bounds behavioral drift.** Without it the compensator is active at nearly every step (19.8 vs. 11.7) and realizes the deepest raw saving (-19.6%), but the paired success difference is the second worst in the table and posture is lower; the floor is the conservative tether that keeps the compensated trajectory recognizable as the commanded task. **Bounding the correction is the robustness mechanism.** Substitution pays the largest paired success cost (-5.0 points here, -6.4 points in an earlier 250-episode five-seed protocol) for savings the bounded correction largely retains. **Annealing is posture insurance, not an energy knob.** Disabling the goal-proximity decay leaves aggregate energy unchanged (-12.2% either way); the component exists to return terminal-approach authority to the base policy, and its effect is target-specific rather than aggregate. **Identification is cheap and degrades gracefully.** With a quarter of the excitation data ($\approx$ one minute of simulation), success stays at parity and the saving shrinks from -12.2% to -6.4% : the binding resource is the energy channel’s accuracy (R^2 0.62 vs. 0.75), precisely the channel the margin guards, and the safety components hold regardless.

Two further design choices are supported by earlier evidence at scale. **Full substitution degrades off the tuning seed:** across the 250-episode five-seed protocol, the substitution variant saved 18.7% energy but lost 6.4 success points against direct (0.664 vs. 0.728). Model error and beyond-horizon posture drift accumulate when the compensator fully owns the command, which is why the correction is bounded. **Margins must be relative and uncertainty-aware:** an absolute energy margin carried from Go2 to H1 *increased* H1 energy by 13.5%, because the H1 model’s energy resolution ($\approx 40\%$ of the H1 per-step baseline) admitted phantom savings; the relative margin of Section 3.3, sized to that uncertainty, is what produces the do-no-harm H1 row of Table 3.

5 Limitations and Future Work

Scope. All results are in simulation, on target reaching, with velocity-commanded locomotion policies across three embodiments. The framework’s assumptions (Section 3.1) characterize command-conditioned continuous control, not reinforcement learning in general: an unconditioned single-task policy exposes no interface to invert over, a task without command redundancy leaves nothing to save, and sparse long-horizon problems hide command consequences from any short-window response model. Within those bounds, the untested breadth is real: end-effector-commanded manipulation and style-commanded character control [Peng et al., 2022] fill the progress/cost/viability schema of Section 3.2 naturally and are the most direct test of generality; for unconditioned policies, distilling a command-conditioned interface first (and then compensating it) is an open path. The recipe requires per-robot identification (four minutes of simulation and minutes of training per robot); ANYmal-D shows that identification is the *only* per-robot component, but it remains a component. Hardware transfer, where energy savings matter most, is the natural next step; the identification recipe applies unchanged wherever the policy can be safely excited.

Model horizon. The 16-step response horizon captures immediate consequences but not slow drift; the restore mode papers over this with measured-height feedback. A multi-horizon or recurrent response model could subsume that branch.

Candidate enumeration. Inversion by enumerating ~ 70 candidates is trivially cheap but coarse; gradient-based inversion through f_ϕ or a learned amortized compensator could refine it, at the cost of the enumeration’s robustness to model error.

Task interface. The current task family is target reaching with a proportional reference; richer tasks (path following, disturbance recovery, velocity schedules) would exercise the compensator’s progress constraint differently and remain untested.

6 Conclusion

We presented VGFC, which compensates a frozen learned continuous-control policy the way classical control compensates a plant: identify the command-response of the closed loop from the policy’s own excitation rollouts, then invert that model at the command interface behind a viability gate that predicts posture safety before execution. On Isaac Lab rough terrain, VGFC systematically improves the energy economy of the strong pretrained Go2 controller it compensates (5–11% lower energy at statistically matched success on every task family evaluated, negative on every evaluation seed); transfers to ANYmal-D with zero re-tuning; and on the H1 humanoid, whose baseline is already efficient, an uncertainty-aware margin correctly leaves the policy nearly untouched. The retrieval-based alternative,

given a full multi-seed study of its own, captures a smaller energy gain and inherits archive-coverage liabilities; and the component ablations map each safeguard onto its role: the gate converts predicted savings into realized ones, the progress floor bounds behavioral drift, and the bounded correction is what keeps model error from buying energy with success. A learned policy’s own experience, we conclude, is most useful not as a library of behaviors to replay but as identification data for the system the policy has become; compensation through that identified model is a cheap, training-free layer that any command-conditioned policy can wear.

Reproducibility

All experiments use public Isaac Lab environments and public pretrained RSL-RL checkpoints. Excitation collection, response-model training, evaluation, and plotting code, together with all result files aggregated in this paper, are available in the project repository.

References

- Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, pages 91–100, 2022. arXiv:2109.11978.
- Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac Gym: High performance GPU based physics simulation for robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2108.10470.
- Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47):eabc5986, 2020.
- Takahiro Miki, Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62):eabk2822, 2022.
- Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.
- Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.
- Bruce A. Francis and W. Murray Wonham. The internal model principle of control theory. *Automatica*, 12(5):457–465, 1976.
- Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1802.06070.
- Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015.
- Karl Pertsch, Youngwoon Lee, and Joseph J. Lim. Accelerating reinforcement learning with learned skill priors. In *Conference on Robot Learning (CoRL)*, 2020. arXiv:2010.11944.
- Marc Peter Deisenroth and Carl Edward Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *International Conference on Machine Learning (ICML)*, pages 465–472, 2011.
- Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018. arXiv:1805.12114.
- Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1906.08253.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1912.01603.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2203.04955.
- Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 7559–7566, 2018. arXiv:1708.02596.
- David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

- Paolo Falcone, Francesco Borrelli, Jahan Asgari, Hongtei Eric Tseng, and Davor Hrovat. Predictive active steering control for autonomous vehicle systems. *IEEE Transactions on Control Systems Technology*, 15(3):566–580, 2007.
- Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Information-theoretic model predictive control: Theory and applications to autonomous driving. *IEEE Transactions on Robotics*, 34(6):1603–1622, 2018.
- Juraj Kabzan, Lukas Hewing, Alexander Liniger, and Melanie N. Zeilinger. Learning-based model predictive control for autonomous racing. *IEEE Robotics and Automation Letters*, 4(4):3363–3370, 2019.
- Jared Di Carlo, Patrick M. Wensing, Benjamin Katz, Gerardo Bledt, and Sangbae Kim. Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9, 2018.
- Lukas Hewing, Kim P. Wabersich, Marcel Menner, and Melanie N. Zeilinger. Learning-based model predictive control: Toward safe learning in control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3:269–296, 2020.
- Emanuele Garone, Stefano Di Cairano, and Ilya Kolmanovsky. Reference and command governors for systems with constraints: A survey on theory and applications. *Automatica*, 75:306–328, 2017.
- Kim P. Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained non-linear dynamical systems. *Automatica*, 129:109597, 2021.
- Tairan He, Chong Zhang, Wenli Xiao, Guanqi He, Changliu Liu, and Guanya Shi. Agile but safe: Learning collision-free high-speed legged locomotion. In *Robotics: Science and Systems (RSS)*, 2024. arXiv:2401.17583.
- Jeonghwan Kim, Yunhai Han, Harish Ravichandar, and Sehoon Ha. Learning Koopman dynamics for safe legged locomotion with reinforcement learning-based controller. *arXiv preprint arXiv:2409.14736*, 2024.
- Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999.
- Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.
- Archit Sharma, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. Dynamics-aware unsupervised discovery of skills. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1907.01657.
- Anurag Ajay, Aviral Kumar, Pulkit Agrawal, Sergey Levine, and Ofir Nachum. OPAL: Offline primitive discovery for accelerating offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.13611.
- Xue Bin Peng, Yunrong Guo, Lina Halper, Sergey Levine, and Sanja Fidler. ASE: Large-scale reusable adversarial skill embeddings for physically simulated characters. In *ACM Transactions on Graphics (SIGGRAPH)*, volume 41, 2022. arXiv:2205.01906.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016.
- Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023. arXiv:2304.13705.
- Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.
- Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6023–6029, 2019.
- Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Robotics: Science and Systems (RSS)*, 2021. arXiv:2107.04034.
- Gabriel B. Margolis and Pulkit Agrawal. Walk these ways: Tuning robot control for generalization with multiplicity of behavior. In *Conference on Robot Learning (CoRL)*, 2022. arXiv:2212.03238.

Mayank Mittal, Calvin Yu, Qinxu Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. arXiv:2301.04195.

Clemens Schwarke, Mayank Mittal, Nikita Rudin, David Hoeller, and Marco Hutter. RSL-RL: A learning library for robotics research. *arXiv preprint arXiv:2509.10771*, 2025.